

## Тестовое задание

Написать скрипт, который бы по непротиворечивому списку требований автоматически составлял систему кодирования команд некоторой RISC-подобной архитектуры.

Список необходимых полей для каждого формата инструкций задаётся в JSON-файле следующей структуры

- `length` - длина инструкции в битах.
- `fields` - список полей которые должны быть закодированы. Поля с одинаковыми именами для всех форматов и инструкций должны располагаться в одном и том же месте.
- `instructions` - массив структур описывающий инструкции. Одна структура описывает однотипные инструкции, поля:
  - `insns` — список инструкций, описывающий все инструкции данного формата
  - `operands` — список операндов инструкции — полей, которые должны быть у каждой инструкции данного формата.
  - `format` — название формата.
  - `comment` — комментарий.

Пример входного JSON-файла:

```
{
  "length" : "25",
  "fields" : [
    { "R0" : "5" },
    { "R1" : "5" },
    { "R2" : "5" },
    { "imm" : ">=8" },
    { "disp" : ">=12" },
    { "code" : ">=3" }
  ],
  "instructions" :
  [
    {
      "insns" : ["add", "sub", "mul", "div", "and", "or", "xor", "lsl", "lsr", "asr", "cmp",
"call"],
      "operands" : ["R0", "R1", "R2"],
      "format" : "alu_instructions",
      "comment" : "3-operand arithmetic and logic instructions, function call"
    },
    {
      "insns" : ["load.b", "store.b", "load.d", "store.d", "load.w", "store.w", "branch", "jump",
"srp", "move"],
      "operands" : ["R0", "R1", "imm"],
      "format" : "memory_access",
      "comment" : "Instructions for reading and writing memory, short branches"
    },
    {
      "insns" : ["branch.cond"],
      "operands" : ["R0", "code", "disp"],
      "format" : "branch",
      "comment" : "branch instruction"
    }
  ]
}
```

Выходной файл — JSON-файл, содержащий массив инструкций, для каждой из которых есть следующие поля:

- `insn` — название инструкции

- **fields** — массив структур операндов с указанием расположения битов операндов
  - **msb** — самый старший бит для поля
  - **lsb** — самый младший бит поля
  - **value** — значение фиксированных битов, для нефиксированных (различные операнды инструкции) указывать «+».

Разрывать поля в кодировке не допускается. Если после раскладки кодировки останутся неиспользованные биты, то за их счёт можно увеличить поля для которых указан минимальный размер (их размер помечен ">="). Если увеличить поля с указанным минимальным размером не получается, то неиспользуемые биты должны быть помещены в именованные поля с названиями RES0/RES1/RES2/....

Пример выходного файла (одно из множества возможных решений см. ниже):

```
...
{
  "insn" : "branch.cond",
  "fields" :
  [
    {"F"      : {"msb" : 24, "lsb" : 23, "value" : "10"}},
    {"CODE"   : {"msb" : 22, "lsb" : 19, "value" : "+"}},
    {"R0"     : {"msb" : 18, "lsb" : 14, "value" : "+"}},
    {"DISP"   : {"msb" : 13, "lsb" : 0,  "value" : "+"}}
  ]
}
...
{
  "insn" : "add",
  "fields" :
  [
    {"F"      : {"msb" : 24, "lsb" : 23, "value" : "00"}},
    {"OPCODE"  : {"msb" : 22, "lsb" : 19, "value" : "0000"}},
    {"R0"      : {"msb" : 18, "lsb" : 14, "value" : "+"}},
    {"R1"      : {"msb" : 13, "lsb" : 9,  "value" : "+"}},
    {"R2"      : {"msb" : 8,  "lsb" : 4,  "value" : "+"}},
    {"RES0"    : {"msb" : 3,  "lsb" : 0,  "value" : "0000"}}
  ]
}
...
```

Литература:

[Википедия - Архитектура MIPS](#)

[osdev.wikia - ARM encoding](#)

Пример решения для указанного выше входного файла.

| Формат alu_instructions |    |        |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |       |   |   |   |   |   |   |
|-------------------------|----|--------|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|-------|---|---|---|---|---|---|
| 24                      | 23 | 22     | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6     | 5 | 4 | 3 | 2 | 1 | 0 |
| F=0                     |    | OPCODE |    |    |    | R0 |    |    |    | R1 |    |    |    | R2 |   |   |   | RES=0 |   |   |   |   |   |   |

  

| OPCODE | Инструкция |
|--------|------------|
| 0x0    | add        |
| 0x1    | sub        |
| 0x2    | mul        |
| 0x3    | div        |
| 0x4    | and        |
| 0x5    | or         |
| 0x6    | xor        |
| 0x7    | lsl        |
| 0x8    | lsr        |
| 0x9    | asr        |
| 0xA    | cmp        |
| 0xB    | call       |

  

Поле RES –  
зарезервировано на  
будущее

  

| Формат memory_access |    |        |    |    |    |    |    |    |    |    |    |    |    |     |   |   |   |   |   |   |   |   |   |   |
|----------------------|----|--------|----|----|----|----|----|----|----|----|----|----|----|-----|---|---|---|---|---|---|---|---|---|---|
| 24                   | 23 | 22     | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10  | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| F=1                  |    | OPCODE |    |    |    | R0 |    |    |    | R1 |    |    |    | IMM |   |   |   |   |   |   |   |   |   |   |

  

| OPCODE | Инструкция |
|--------|------------|
| 0x0    | load.b     |
| 0x1    | store.b    |
| 0x2    | load.d     |
| ...    | ...        |
| 0x9    | sip        |
| 0xA    | move       |

  

| Формат branch |    |      |    |    |    |    |    |    |    |      |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|---------------|----|------|----|----|----|----|----|----|----|------|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| 24            | 23 | 22   | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14   | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| F=2           |    | CODE |    |    |    | R0 |    |    |    | DISP |    |    |    |    |   |   |   |   |   |   |   |   |   |   |

  

|                       |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|-----------------------|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|
| F=3 – зарезервировано |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|-----------------------|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|

Данный пример не является выводом программы, это визуализация возможного внутреннего состояния программы.

Пример кодирования инструкций для указанного выше примера решения:

Пример – инструкция branch \$29, \$8, 0x2A

F=1

OPCODE=6

IMM=0x2A

R0=29(=0b11101)

R1=8(=0b01000)

|     |    |        |    |    |    |    |    |    |    |    |    |    |    |     |   |   |   |   |   |   |   |   |   |   |
|-----|----|--------|----|----|----|----|----|----|----|----|----|----|----|-----|---|---|---|---|---|---|---|---|---|---|
| 24  | 23 | 22     | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10  | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| F=1 |    | OPCODE |    |    |    | R0 |    |    |    | R1 |    |    |    | IMM |   |   |   |   |   |   |   |   |   |   |
| 0   | 1  | 0      | 1  | 1  | 0  | 1  | 1  | 1  | 0  | 1  | 0  | 1  | 0  | 0   | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 0 |

Пример – инструкция add \$1, \$2, \$3

F=0

OPCODE=0

R0=1

R1=2

R2=3

|     |    |        |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|-----|----|--------|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| 24  | 23 | 22     | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| F=0 |    | OPCODE |    |    |    | R0 |    |    |    | R1 |    |    |    | R2 |   |   |   | 0 | 0 | 0 | 0 |   |   |   |
| 0   | 0  | 0      | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 0  | 1  | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 0 |